

The Vektorgeist Method (VGM): Sovereign, Local, Self-Understanding AI

Alexander Parnell · Vektorgeist

Position paper / program overview, 2026 · The Vektorgeist Method (VGM)

This paper names and frames a research program. It is the umbrella over a series of focused works — the first of which, on the afferent self-model **AGM**, is published separately. A systems paper on the sovereign local stack follows. This overview states the shared commitments and how the pieces fit.

ABSTRACT

Most capable AI today is rented, opaque, and unaccountable: it runs on someone else's servers, cannot faithfully explain itself, and is governed by whoever owns the endpoint. We describe the **Vektorgeist Method (VGM)** — a methodology for building AI that is the opposite on each axis: it runs **locally** on hardware you own; it is **model-free wherever a deterministic algorithm suffices**, so its behavior is inspectable rather than inferred; it grades generated work by an explicit rule — "**model proposes, oracle disposes**" — rather than trusting fluency; and it observes itself through an **afferent-only self-model** that reports but never steers. VGM is not one artifact but a program: a family of small, sovereign components that together make a system a person can own, inspect, and trust. This paper states the four commitments, maps them onto the current body of work, and locates the program's humanistic premise — that nothing is what it is in isolation — in a companion work of nonfiction.

1 The position

There is a default shape to modern AI, and it is worth naming so it can be argued with. The default is: a very large model, trained and hosted by a large organization, reached over a network, priced per call, updated without notice, and fundamentally unauditable by the person using it. It is powerful and it is convenient, and it also means the most intimate tool many people now use — the thing they think out loud to — is something they rent and cannot see inside.

VGM takes the opposite position on each of those properties, not as ideology but as engineering choices with consequences. The consequences are the point: an AI you run locally cannot be silently changed or revoked; an AI built from small deterministic parts can be inspected where it counts; an AI that grades its own output against an explicit standard fails loudly instead of confabulating confidently; and an AI that watches itself — honestly, without the power to act on what it sees — can be understood in real time. None of these is exotic. Together they describe a different kind of system: one that belongs to its operator.

2 The four commitments

VGM is defined by four commitments. They are independent enough to adopt one at a time and coherent enough to reinforce each other.

2.1 Local-first / sovereign

The system runs on hardware the operator controls — down to a single consumer GPU — and does not depend on a remote endpoint to function. Sovereignty here is concrete: no per-call metering, no remote kill switch, no data leaving the machine as a condition of use. Capability is traded against control deliberately, and the method's wager is that a well-composed small system, owned outright, beats a larger one you merely borrow for the uses that matter most.

2.2 Model-free where a deterministic algorithm suffices

A large model is the right tool for open-ended language and judgment; it is the wrong tool for tasks a plain algorithm does exactly. VGM pushes deterministic, inspectable code into every role where such code exists — retrieval, routing, gating, measurement — and reserves the model for what genuinely needs it. The result is a system whose skeleton is auditable and whose model-dependence is minimized rather than maximized. Less of the behavior is inferred; more of it can simply be read.

2.3 "Model proposes, oracle disposes"

Where a model does generate something that must be correct — code, a graded answer, a structured artifact — VGM does not trust the generation on the strength of how plausible it sounds. A separate, explicit **oracle** decides whether the proposal passes: tests that run, a checker that verifies, a rule that holds or does not. The model is a proposer; the verdict comes from something that can actually be wrong out loud. This converts fluent-but-false — the failure mode of generation — into a caught error.

2.4 Afferent self-modeling

A VGM system can observe its own behavior through a self-model that is strictly **afferent** — it senses and reports upward to the operator, but is architecturally forbidden from re-entering the answer, becoming memory, or gating any component. This is the subject of the program's first paper (§3). The commitment matters at the program level because it draws the line between a system that understands itself and one that manages you: the self-knowledge is real and the self-knowledge is powerless, on purpose.

3 The program

The commitments above are realized as a family of focused components. Each is small; the method is in how they compose.

AGM — the afferent self-model (flagship, published). AGM is a self-model of three organs — Aitía (cause), Gut (instinct), and Metron (measure) — that sits beside an agent and produces a faithful, human-readable readout of *why* it did something and *whether it is drifting*, under the strict afferent-only rule described in 2.4. AGM is the clearest single expression of the VGM stance: self-understanding without self-steering. It is presented in full in the companion paper, *AGM: An Afferent Self-Model for Interpretable, Self-Monitoring AI Agents* (Parnell, 2026).

Connection memory. Memory in a VGM system is treated as *connected information*, not a flat log: a graph of facts and relationships, retrieved by explicit, inspectable ranking rather than by a single opaque similarity call. This is the model-free commitment (2.2) applied to memory — retrieval you can read and correct. (Described here at the level of principle; the reference implementation is a private product.)

The sovereign local stack (systems track, forthcoming). A self-hostable stack that assembles these commitments into a running whole — a gateway, the connection memory, retrieval, and containment boundaries — targeted at ordinary hardware. This track is the subject of a separate systems paper and is described there; this overview only locates it in the program.

The grading discipline. "Model proposes, oracle disposes" (2.3) is realized as a general pattern: generation is always paired with an explicit disposer, whether that is a test suite for code or a checker for a structured task. It appears across the program rather than in one component.

4 Why it is built this way

The commitments are not neutral engineering taste; they encode a view about the relationship between a person and a machine that thinks alongside them. A tool you cannot see inside, cannot run without permission, and cannot hold to account is not neutral either — it simply hides its politics in its defaults. VGM

makes the opposite defaults: inspectable over inferred, owned over rented, powerless self-knowledge over hidden control. The claim is not that the sovereign system is always more capable. It is that for the uses that matter — the private, the load-bearing, the ones you would not want silently revised — ownership and legibility are worth more than the last increment of raw capability.

5 The humanistic premise

Underneath the engineering is a single idea, and it is older than any of the machinery: **nothing is what it is in isolation; a thing becomes what it is through what it is connected to.** A memory is information joined to other information. An identity is a network of experiences. Understanding is the tracing of connections. The technical program is, in a sense, this idea taken literally — memory built as a graph of relations, a self-model built from how behavior connects across time.

That premise is argued at length, for a general reader and without any of the engineering, in a companion work of literary nonfiction, *Your Past Loves You* (Parnell, 2026). It is not a book about AI; AI appears in it only as a mirror for understanding the human. But it is the plainest statement of the thesis the method is built on, and it is offered as the program's front door for readers who would never open a systems paper.

6 The paper series

VGM is a pipeline, not a single release:

1. **AGM** — the afferent self-model. *Published (Parnell, 2026).*
2. **The sovereign local stack** — the systems paper. *Forthcoming.*
3. **This overview** — the program and its commitments.

Further coined components fold in as the program continues; a running index is maintained at vektorgeist.com/research, with each paper archived to a persistent DOI. The method is meant to accumulate — to become a citable body of work under one name — rather than to arrive all at once.

7 Conclusion

The Vektorgeist Method is a bet that the right response to AI you cannot see inside is not better prompting but a different architecture: local so it is yours, deterministic where it can be so it is legible, oracle-graded so it fails honestly, and afferently self-aware so it can be understood without being able to steer you. Each piece is modest. The wager is that assembled, and owned, they add up to a kind of AI worth trusting — because you can see it, and because it belongs to you.

References

- Parnell, A. (2026). *AGM: An Afferent Self-Model for Interpretable, Self-Monitoring AI Agents*. Preprint (Zenodo). The flagship pillar of VGM.
- Parnell, A. (2026). *Your Past Loves You*. Literary nonfiction. (The program's humanistic premise: connection as the substrate of memory and identity.)

Author: Alexander Parnell. Correspondence: Vektorgeist. Intended archival venue: Zenodo (DOI), with the program indexed at vektorgeist.com/research. The sovereign-stack systems paper is described in its own forthcoming work and is intentionally not detailed here.